Egocentric Human Demonstrations for a Bimanual Robot
Part I: Three-Cube Stacking

Jeong Hwan Lee
Technical report, version 4.1, 12 October 2026 alnrg2arg@gmail.com
Part II covers single-arm approach data. Code, results and sample data are available on request.

***Abstract*—We study how far natural egocentric human demonstrations, recorded with HandUMI, an open-source hand-worn UMI-style gripper designed and published by BrikHMP18 (not our design), can be turned into training data for a bimanual robot policy. We collected 349 bimanual demonstrations of a three-cube stacking task with six stacking orders balanced at collection time (51–64 episodes per order), using per-hand wrist fisheye cameras, IMUs and jaw encoders plus a fixed head camera. A multi-stage pipeline (MASt3R-SLAM wrist poses with per-episode visual–inertial scale, retargeting to a 6-DoF robot arm, and synchronisation, workspace, metric and inverse-kinematics gates) keeps 259 episodes, or 561 bimanual 65-frame segments. On these we fine-tune a 0.9B-parameter vision-language-action model (X-VLA) to predict 30-step joint-space action chunks, and evaluate it offline on 26 held-out episodes. The median held-out TCP error is 1.5–5.7 mm over a 0.2–1.2 s horizon. This figure is dominated by static phases: on the 23% of samples in which the target moves at least 20 mm, the error at 1.2 s is 39.9 mm, direction cosine 0.90, and the policy reduces joint error by 24% relative to a zero-motion predictor. Swapping the order instruction changes the predicted motion measurably but by only $\sim 8\%$ of its magnitude, and the true instruction predicts the human motion no better than the five wrong ones, so the model makes no task-useful use of the order instruction in this evaluation. Two follow-ups address the main bottleneck. Replacing the single-anchor retarget with multi-seed trajectory-window retrieval raises the usable fraction of candidate segments from 36.5% to 56.1% and brings the retargeted postures much closer to real robot postures; a 60-episode robot-like re-collection is used to test whether collection style changes retarget feasibility; it does not appreciably, a negative, diagnostic result. Fine-tuned on 150 robot demonstrations, the ego-pretrained model has a 10% lower geo score than an otherwise identical model trained from scratch (95% CI 5.6–14.7%), but only on episodes contained in both training sets. This version adds a third stage that removes joint-space retargeting altogether. Whole human episodes are converted to relative Cartesian end-effector actions in the same 20-dimensional layout as our robot data, with no workspace or IK filtering (330 episodes, 48,411 training rows after a pose-jump filter). An audit of the public X-VLA base shows that only 8 of its 30 domain soft-prompt slots were ever trained, which changes how earlier runs should be read. On robot camera frames the ego-only model predicts no useful motion direction (median cosine -0.05/-0.03 versus $+0.81$/$+0.77$ for a robot-trained model); swapping images and states between domains locates the gap in the wrist images, whose sharpness differs by about 30×. Used as an initialization for robot fine-tuning, the ego model starts at a 3.9× lower training loss, but the advantage shrinks to a tie by 60k steps of an otherwise identical run. Version 4 adds a low-data ablation: three otherwise identical fine-tuning runs on only 30 robot episodes, from the base model and from ego pretraining checkpoints at 100k and 300k steps. Both ego initializations start at less than half the scratch training loss; the advantage is gone by about 100k steps (100k-step pretraining) and about 200k steps (300k-step pretraining), and all three runs overfit 30 episodes heavily. None of them has yet been run on the robot. An ego-initialized model fine-tuned on 312 robot episodes completed full three-cube stacks in closed loop, but trials were not counted, so we report a demonstration, not a success rate. No experiment tests new spatial layouts or unseen stacking orders; we state explicitly which claims this data can and cannot support. The single-arm approach work that was Sec. XII of version 3 now forms Part II. Version 4.1 adds a description of the capture hardware: the upstream HandUMI design we built, the parts we changed, and the IMU we added.**

I. Introduction

Collecting robot demonstrations is often expensive, requiring access to a physical robot, an operator and task-specific teleoperation infrastructure. Egocentric human demonstrations can be collected faster, with less hardware, in natural settings, and hand-worn grippers such as UMI [1] make the human's end-effector look like the robot's, reducing the embodiment gap at the gripper. Whether such data transfers to a specific robot depends on a chain of steps that is rarely audited end to end: sensor calibration, metric pose recovery from a moving wrist camera, retargeting into the robot's kinematics, and the filtering that decides which human motion is kept at all. Each step can silently change what a policy ends up learning.

This report documents one complete pass through that chain on real data and asks a deliberately narrow question: *after all processing, how much robot-executable action structure does a pretrained vision-language-action (VLA) model learn from retargeted egocentric human demonstrations, and does it use the task-order instruction?* Our contributions are:

- a build record of the hand-worn capture hardware: the upstream design, three modified parts, an added wrist IMU with its firmware protocol, and the device-identity pitfalls (Sec. III);
- a 349-episode bimanual egocentric dataset of three-cube stacking with six balanced stacking orders, with full sensor calibration and per-episode metadata (Sec. IV);
- a documented processing funnel from raw human video to robot joint-space segments, reporting what each gate removes (Sec. V);

- an offline evaluation of a 0.9B-parameter VLA fine-tuned on the result, including a motion-gated analysis showing that aggregate errors are dominated by static phases, and a prompt-swap test of instruction use (Secs. VII–IX);
- follow-up experiments on the retargeting bottleneck, robot-like re-collection and ego-to-robot fine-tuning (Sec. X);
- a Cartesian conversion of whole episodes that matches the robot's action contract, with the pose-jump failure it exposed (Sec. XI), and transfer diagnostics that separate visual, state and initialization effects (Sec. XII);
- a low-data ablation of ego pretraining length on 30 robot episodes (Sec. XIII).

We are explicit about scope. The evaluation is open-loop on held-out *episodes*; it is not a closed-loop robot test, and because layouts were not recorded and all six orders appear in training, it measures neither spatial nor task-order generalization. Sections I–X are unchanged in substance from version 2 (30 September 2026) and Sections XI–XII from version 3 (4 October 2026), apart from run status; Sec. XIII is new in version 4 and Sec. III in version 4.1 (12 October 2026). Version 3 also contained a single-arm approach study; it has grown into a separate report (Part II), which reuses the device, pose pipeline and action contract described here. Sec. XV lists the experiments needed for the claims we do not make.

II. Related and Motivating Work

Portable demonstration interfaces. UMI [1] records handheld gripper demonstrations with a wrist fisheye camera and recovers poses with ORB-SLAM3 against a per-scene map; DexCap [2] and related systems target dexterous hands. Our device is HandUMI [16], an open-source hand-worn UMI variant for parallel-jaw grippers with a servo-encoder jaw. We replace its VR tracking with a wrist IMU and add a fixed head camera (Sec. III). **Learning from egocentric humans.** EgoMimic [3] and MimicPlay [4] co-train robot policies on human egocentric video and robot data; large egocentric corpora such as Ego4D [5] motivate the scaling argument. These works report closed-loop gains; we instead audit what survives processing and what a model learns offline, which is a prerequisite for interpreting such gains. **Policies.** Action-chunking imitation (ACT [6], Diffusion Policy [7]) and VLA models (OpenVLA [8], π_0 [9], X-VLA [10]) predict multi-step action chunks from images and language. We use X-VLA through LeRobot [11] because its flow-matching [12] head and soft prompts accommodate a new embodiment with a small dataset. **Pose recovery.** MASt3R-SLAM [13] provides dense monocular SLAM from a 3D reconstruction prior; we combine it with IMU-based scale estimation, calibrated with Kalibr [14].

III. HandUMI Hardware

Attribution. The capture device is *not our design*. HandUMI was designed and published as open-source hardware by BrikHMP18 (https://github.com/murobotics-ai/handumi-hw, Apache-2.0) [16], together with its software (https://github.com/murobotics-ai/handumi-sw) [17]. We built it from upstream commit `e58de33`. Our own hardware work is limited to three modified parts, a wrist IMU added in place of the upstream VR tracking, and a fixed head camera. Our `umihand` repository is a build copy of the upstream files plus these additions, not an original design.

Upstream HandUMI. HandUMI is an open-source, hand-worn variant of UMI [1] for robot arms with parallel-jaw grippers, published by BrikHMP18 under Apache-2.0 in the `murobotics-ai` GitHub organization (https://github.com/murobotics-ai/handumi-hw; first released on 20 August 2026 as `robonet-ai/handumi-hw`, which now redirects there). UMI removed the robot from data collection, and YUBI [18] applied that idea to a finger-driven gripper. HandUMI is the parallel-jaw counterpart. It mounts on the thumb and the index and middle fingers and closes with a natural pinch. Its gripper tips are detachable (AgileX Piper, ARX X5, Dream Gripper, WidowX AI and the UMI gripper), so one body serves several robots. Upstream records three signals. Wrist SE(3) comes from a VR headset (PICO 4 Ultra or Meta Quest 3) and its controllers, which a printed controller support holds on the device. Jaw width is read directly from a Feetech servo encoder, not inferred from fiducials or image segmentation. The third signal is wrist-view video. The finger cradles were modeled on a 3D scan of the author's hand. A companion repository, `handumi-sw` [17], records, visualizes and retargets the demonstrations to bimanual arms.

What we took from it. The hardware is commit `e58de33`, which uses an Arducam fisheye camera. Upstream has since priced the BOM per pair and moved to an IMX335 camera (commit `fc7462a`); we did not adopt that change. We forked `handumi-sw` at commit `33cc437` for an early ego-bimanual path to the reBot B601. The fork added Quest 3S tracking, AprilTag end-effector tracking, Feetech jaw telemetry, a reBot B601 robot profile and tracking-quality gates. None of the demonstrations in Parts I and II were recorded with that path. They come from our own collector, which keeps the upstream body and jaw encoder but replaces VR tracking with the wrist camera and an added IMU (below). The print package, our modified parts, the STEP sources, the BOM and the IMU firmware are kept as a separate hardware repository (`umihand`) and are mirrored in `hardware/` and `collection/firmware/` of the code repository.

Mechanics and print. The unit is a hand-support base with thumb and index/middle cradles, a fisheye camera support, and a crank linkage (MR63ZZ bearings, LM4UU linear bearings on a 4 mm shaft) that couples the finger closure to a detachable gripper tip and to a servo read as the jaw encoder. One unit is 12 printed parts. The package has 9 plates and 26 parts for both hands plus a generic tip, about 9 h and 250 g of PLA (0.20 mm layers, 2 walls, 20% infill, Bambu X1C), about 4 h and 110 g per hand. Supports are needed only on the controller support. The cradles come from the upstream author's hand scan, so a fit-check plate (cradles and base, about 1 h) is printed first for each hand. Tight holes are fixed with $+0.05$ to 0.1 mm XY hole compensation, not by scaling parts. The upstream STLs

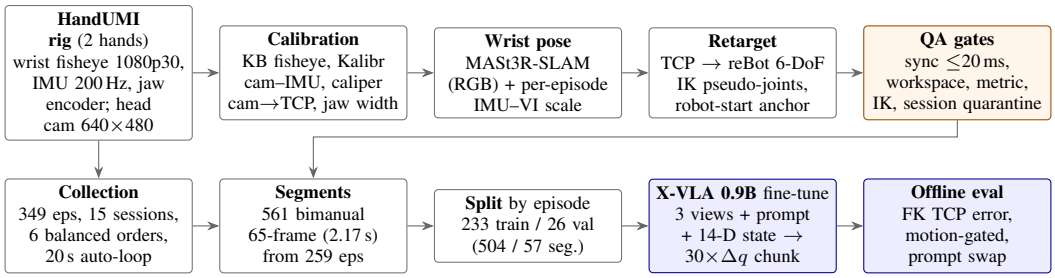


Fig. 1. System and data pipeline. Orange: filtering stages whose losses are reported in Table I; blue: learning and evaluation. No stage uses robot rollouts.

have negative z, which the slicer CLI rejects; we only translate them. The tip in the package is the AgileX-Piper parallel-jaw tip, kept as a temporary tip for mechanism and pinch tests; no reBot-specific tip has been designed yet.

Electronics. Each unit carries an Arducam 160° fisheye USB camera (1920×1080 MJPG, 30 fps), a Feetech STS3215 servo read as the jaw encoder through a Waveshare serial-bus adapter (CH343, 1 Mbaud, servo id 1, needs its 12 V supply before the servo answers), and a wrist IMU. The upstream BOM is about US$110 per unit.

Deviations from upstream. (i) No VR tracking: upstream takes wrist SE(3) from a PICO/Quest controller, we recover it from the wrist camera plus IMU (Sec. V). The controller support is therefore replaced by our connector. (ii) An added IMU per unit: a Teensy 4.1 with an ICM-42688-P on SPI. The sensor has no 400 Hz rate, so it runs at 200 Hz; the data-ready interrupt time-stamps each sample on the device, and packets (40-byte payload, CRC-16) go over USB CDC with a 1 Hz status packet that reports dropped samples and buffer high-water. The host stamps arrival times and fits the device-to-host clock offline, so USB jitter does not enter the timeline. The board must be flashed as USB Serial; a RawHID build shows no serial port and looks dead. (iii) A fixed Logitech C922 head camera at 640×480, autofocus off with a fixed focus, because autofocus changed f_x between sessions (417, 531, 501 px).

Our part changes. Three parts were edited in Shapr3D (Fusion 360 and FreeCAD were tried and rejected), shown in Fig. 2: a new left controller-support connector (8 September, 85.7×21.4×42.1 mm, printed twice), whose clamp plate was then lengthened by 10 mm (14 September, depth 21.4→31.4 mm; 15 mm and three z+30 mm variants were rejected); a camera mount with a second window (8 September); and a servo cover with a cable opening widened to almost its full width (11 September). Outlines and mounting points are unchanged.

Device identity. Both Arducams report the same USB serial (`UC684`), so the recorder matches them by product name and refuses to record unless exactly one device matches each name; IMUs are matched by Teensy USB serial and jaws by adapter serial plus servo id, never by port order. One jaw crosses the encoder's 4095→0 seam, so jaw normalization is circular. A 12.2 V supply sets the servo's "voltage" error bit against its 12.0 V limit; this is telemetry, not a fault. Part I uses both units (profile `handumi_v1`); Part II uses the right unit only.

IV. Dataset and Task Design

Recording. Each hand wears a HandUMI unit (Sec. III): a 1920×1080, 30 fps fisheye camera, a 200 Hz IMU and a jaw encoder sampled at 100 Hz. A fixed head-mounted Logitech C922 records a 640×480, 30 fps view of the table; this is the global view given to the policy (Fig. 3a). All streams are written to one MCAP file and three videos per episode.

Task. Three 50 mm cubes (red, blue, purple) are stacked on a fixed plate in one of six orders; the instruction names the order, e.g. "Stack the blue cube on the bottom, purple cube in the middle, and red cube on the top, on the plate." An auto-loop records a 20 s take after spoken cues and gives a 10 s reset in which the operator re-places the cubes by hand. The next order is the least-collected one (target 50 per order), so orders are *balanced*, not randomized. The cube layout varies between episodes through the manual reset, around a nominal layout matched to our robot workspace, but it was not recorded; there is no per-episode layout label.

Collection. One operator recorded 349 episodes in 15 sessions over three days (15–17 September 2026), 2.20 h in total (median episode 23.05 s). Per order: RBP 64, RPB 62, BRP 61, BPR 56, PRB 55, PBR 51 (Fig. 3b). Episodes from ten early sessions (55 episodes) are tracked as a separate provenance domain ("early") from the main 294, because their gripper anchors were derived afterwards; after filtering the two domains behave alike.

Calibration. Wrist cameras use a Kannala–Brandt fisheye model (left f_x=789, right f_x=791 px). Kalibr camera–IMU calibration gives time offsets of 30.2 and 32.8 ms with mean reprojection errors of 6.0 and 5.6 px, which is high and one reason pose accuracy is validated separately (Sec. V). The camera-to-TCP transform is a caliper measurement, $t = [0, 55.3, 122.5]$ mm with a fixed axis permutation, following UMI's use of a CAD constant; it was measured on one unit and applied to both. The jaw-width scale (aperture 67.5 ± 5 mm) is estimated from video of 50 mm-cube grasps; a caliper check is pending.

V. Processing and Splits

Wrist poses. UMI localizes all demonstrations in one ORB-SLAM3 map per scene. On our data this failed: maps built before a scene change did not relocalize, and on a marker

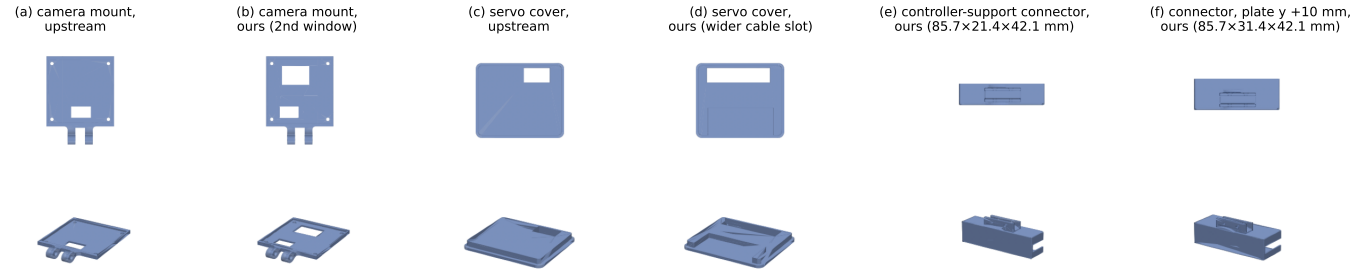


Fig. 2. Our changes to the upstream HandUMI parts (renders of the printed meshes). (a,b) camera mount: a second window cut above the existing one; (c,d) servo controller cover: the cable opening widened; (e) the new left controller-support connector that replaces the upstream VR-controller support; (f) its clamp plate lengthened by 10 mm, the current version.

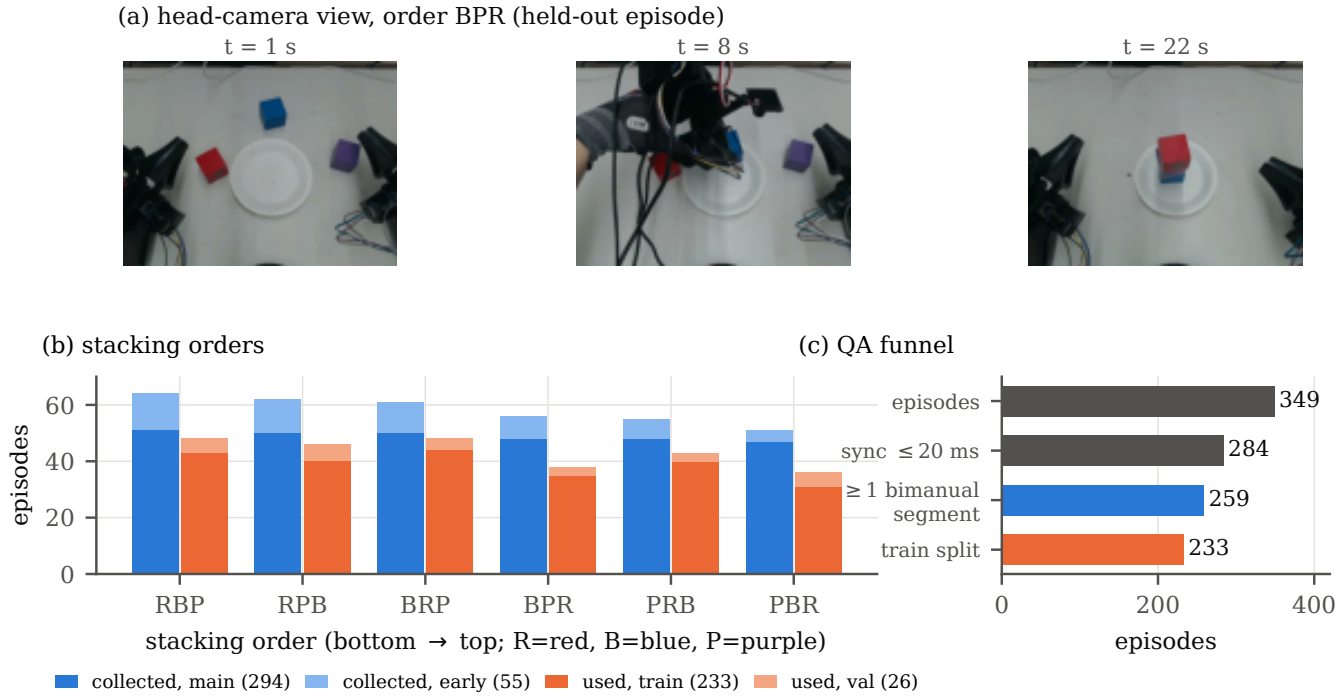


Fig. 3. Dataset design. (a) Head-camera frames at 1, 8 and 22 s of a held-out episode (order BPR). (b) Episodes per stacking order, as collected (main/early domains) and after processing (train/val). (c) Episode-level QA funnel.

benchmark (below) its tracks cover a median of 3% of each demonstration, with frequent position reversals (median reversal rate 0.24 versus 0.07 for the method we adopt). We therefore run MASt3R-SLAM on the RGB wrist video alone and recover metric scale per episode from the IMU (visual–inertial fit over a 15-frame window). On a separate benchmark of 14 ArUco-marker episodes (28 hand trajectories; 10 held out), run on the same GPU as the dataset, this gives median tracking coverage 1.00 versus 0.03 for ORB-SLAM3 with a stored map, scale error p50/p90 of 4.6/17.0% (versus 18.5/23.8% for one global constant), rotation error p50/p90 0.84/2.13° and a UMI-style 16-step relative position error p90 of 11.3 mm. The 349 episodes themselves have no ground truth.

Retargeting. Each TCP trajectory is mapped to a reBot B601 6-DoF arm by inverse kinematics on its URDF. Absolute workspace geometry is not recoverable across episodes, so each segment is re-anchored at a fixed robot start pose (the median active pose of our robot teleoperation data); motion is transferred, absolute cube positions are not. The jaw encoder becomes a binary gripper command.

Gates. Table I lists the funnel. 65 episodes fail the $\leq$20 ms synchronisation requirement for a full 65-frame window; the remaining 284 yield 1,306 candidate bimanual segments, of which 677 leave the robot workspace, 50 fail the metric-scale check and 18 fail IK, leaving 561 (43.0%). Two sessions (6 episodes) were quarantined for an IMU–camera rotation inconsistency; none contributed a segment. The training unit is a 65-frame (2.17 s) segment; 47% of segments start in the

TABLE I
PROCESSING FUNNEL (COUNTS RECOMPUTED FROM THE CENSUS FILES IN THE REPOSITORY).

Stage	Episodes	Segments
Recorded (KEEP)	349	–
Pose + gripper exported (698 tracks)	349	–
Sync $\leq$20 ms, $\geq$65-frame window	284	1,306
− workspace / metric scale / IK		−677 / −50 / −18
$\geq$1 bimanual segment (training pool)	259	561
train / val (by episode, seed 0)	233 / 26	504 / 57

first 5 s of an episode.

Split. The 259 episodes with at least one bimanual segment are split at random by episode (seed 0, 10%): 233 train / 26 val, i.e. 504/57 segments and 32,760/3,705 frames. No episode spans both sides. The split is not stratified; validation has 3–6 episodes per order, 22 main / 4 early, from 9 of the 13 non-quarantined sessions.

VI. MODEL AND TRAINING

We fine-tune X-VLA [10] (LeRobot implementation) from the public `xvla-base` checkpoint: a Florence-2 vision-language encoder [15] and a 24-layer transformer policy (hidden 1024, 16 heads, 32 soft prompts) with a flow-matching action head (10 denoising steps); 879M parameters, all trainable. Inputs are the three views (head, left wrist, right wrist; resized to 224 px), the order instruction and a 14-D state (6 joint angles and a binary gripper per arm).

Action target. For state time t the model predicts a 30-step chunk $a_i = q_{t+5+i} - q_t$, $i = 0, \ldots, 29$, of cumulative joint displacements of the retargeted pseudo-joints (a 5-frame lead, so the chunk covers 0.2–1.17 s ahead), plus the absolute gripper command. Two auxiliary terms tie joint space to the measured human motion: (C) the relative TCP position of the measured trajectory, predicted in spare action dimensions (weight 2.0), and (D) a forward-kinematics loss between $\mathrm{FK}(q_t + \hat{a}_i)$ and the measured TCP position (weight 20). Targets are the measured human transitions, never relabelled robot commands.

Optimization. 300k steps, batch 4, AdamW (lr 10^{-4}, weight decay 10^{-4}, 1k warm-up, cosine decay to 2.5×10^{-6}), gradient clipping 10, fp32, one RTX 4090. A checkpoint is saved every 10k steps. The checkpoint selection rule, minimum geo score (below) on all validation samples, was fixed before training.

VII. EVALUATION PROTOCOL

Evaluation is open-loop on the 26 held-out episodes: from each of the 57 validation segments we take 7 start frames (every 5 frames in 0–30), giving 399 samples and 798 arm-samples, and compare the predicted chunk against the recorded future. For horizons $k \in \{1, 4, 8, 16, 30\}$ we report the TCP position error between $\mathrm{FK}(q_t + \hat{a}_k)$ and the measured TCP (median, mm), the joint MAE and that of a *zero-motion* predictor ($\hat{a} \equiv 0$), and the cosine between predicted and measured TCP displacement. The *geo score* is the mean of the median TCP error over the five horizons. Because most samples are near-static, we also report every metric on the *moving* subset, arm-samples whose measured TCP moves $\geq$20 mm by k=30 (187 of 798, 23.4%). This gate was added after the 60k checkpoint had been evaluated and is used for reporting only.

Prompt swap. To test instruction use, we query the final checkpoint on every sample with each of the six order instructions using the same noise seed, and with the true instruction under two further seeds. The spread of the six predictions measures the instruction's effect; the spread across seeds is the sampling-noise floor.

What the split does not test. Validation episodes share operator, scene, sessions and all six orders with training, and layouts are unlabelled. The results therefore measure within-distribution generalization to new *episodes*, not to new layouts, orders or scenes.

VIII. RESULTS

Table II and Fig. 4 summarize the evaluation. Four findings stand out.

(1) Aggregate errors mostly reflect static phases. The geo score reaches 2.78 mm at 160k and 2.88 mm at 300k and is flat after about 120k (Fig. 4a). On moving samples, however, the error is an order of magnitude larger: 11 mm at the shortest horizon and 40 mm at k=30, with a p90 of 90 mm; the moving-subset geo score is 23.0 mm versus 2.9 mm on all samples.

(2) On moving samples the policy predicts the direction of motion and beats a no-motion baseline. The direction cosine rises from 0.72 at k=1 to 0.90 at k=30, and joint MAE is 13–32% lower than the zero-motion predictor across horizons (6.59 versus 8.68° at k=30; Fig. 4b). The model captures the direction of the upcoming motion, not only its presence.

(3) On all samples, the policy is worse than predicting no motion. Joint MAE is about twice the zero-motion MAE at every horizon (0.30 versus 0.15° at k=1): during static phases the policy predicts small spurious motion, and the median predicted-to-measured displacement ratio is 1.41 at k=1. At k=30 the spread of predictions is 0.77 of the spread of targets, so the long horizon is somewhat under-dispersed rather than collapsed.

(4) The order instruction changes predictions but does not improve them. In the prompt swap (Fig. 4c), the six instructions move the predicted TCP by a median of 4.8 mm at k=30 on moving samples, about 50× the sampling-noise spread (0.09 mm) but only 8% of the predicted displacement (62 mm). The true instruction is not closer to the recorded motion than the wrong ones: its error is 39.9 mm versus 39.5 mm for the mean wrong instruction, and its mean rank among the six is 3.55 (chance 3.5; rank 1 in 16% of cases, chance 17%), with the same picture at every horizon and on all samples. The prediction with the true instruction and the original seed reproduces the reference evaluation to within

TABLE II

OFFLINE HELD-OUT RESULTS (26 EPISODES). TCP ERROR: MEDIAN FK POSITION ERROR (MM); MAE: JOINT MAE (DEG), WITH THE ZERO-MOTION PREDICTOR IN PARENTHESES; COS: TCP DIRECTION COSINE. 160K IS THE PRE-REGISTERED SELECTION, 300K THE FINAL STEP.

		horizon k (steps of 1/30 s after a 5-frame lead)				
Subset	Metric	1	4	8	16	30
All (798)	TCP err., 160k	1.3	1.7	2.2	3.1	5.7
	TCP err., 300k	1.5	1.8	2.2	3.2	5.7
	MAE, 300k	0.30 (0.15)	0.34 (0.19)	0.42 (0.25)	0.57 (0.36)	0.97 (0.51)
Moving (187)	TCP err., 160k	11.2	15.0	19.9	29.2	39.1
	TCP err., 300k	11.2	15.0	19.4	29.3	39.9
	MAE, 300k	2.02 (2.32)	2.68 (3.36)	3.29 (4.38)	4.30 (6.34)	6.59 (8.68)
	cos, 300k	0.72	0.80	0.84	0.86	0.90

10^{-5} mm. A caveat is that much of a 2.2 s segment is order-independent (reaching, lifting); a test restricted to choice points (the first reach of an episode) needs more such samples than the 26 validation episodes provide.

Per-order breakdowns are in the repository; with 3–6 validation episodes per order they are too small to rank orders, and we do not interpret differences between them.

IX. FAILURE ANALYSIS

Weak use of the task-order instruction. The order prompt measurably changes the prediction, but this variation does not improve agreement with the recorded motion (finding 4). In this dataset the order is also partly visible from the scene once stacking has begun, so the instruction is only necessary at the start of an episode; nothing in training forces the model to use it.

Static-phase drift. The dominant error mode is non-zero motion predicted while the human is still. It is the reason the all-samples MAE exceeds the zero-motion baseline, and on a robot it would appear as creep before grasps or after placement. One plausible contributor, not yet tested, is residual pose noise in near-static windows, which makes "stationary" targets noisy.

Long-horizon under-dispersion. Predictions at k=30 vary less than the targets (ratio 0.77) and the p90 error on moving samples reaches 90 mm, so fast or unusual motions are pulled towards the mean.

Data lost before learning. 57% of the candidate segments are removed; the workspace gate accounts for 677 of the 745 removals (91%), and in 531 of those (78%) only one arm leaves the robot workspace. Because the gate removes motions by their extent, the kept set is likely biased towards compact motions; we have not yet quantified this bias.

Upstream failures that shaped the design. ORB-SLAM3 with a stored map failed to relocalize across scene changes and produced position jitter; a camera-orientation convention error in an earlier export corrupted rotations until it was found by a frame-regression test; and 65 episodes (19%) fail synchronisation. Each was found by an explicit gate, not by training loss.

TABLE III

RETARGETING VARIANTS ON THE 1,229 CANDIDATE SEGMENTS OF THE 259 EPISODES, MEASURED AGAINST 120 HELD-OUT ROBOT EPISODES. NN: NEAREST-NEIGHBOUR JOINT-SPACE DISTANCE TO ROBOT POSTURES (RAD, P50/P90); W1: WASSERSTEIN DISTANCE OF $|\Delta q|$ PER STEP.

Variant	Usable	NN p50 / p90	Start clusters	W1 $\lvert\Delta q\rvert$
v1 single anchor	36.5%	1.49 / 1.55	1	0.00197
v2-K multi-seed IK	66.6%	1.21 / 1.60	–	0.00275
v2-Kr + posture penalty	67.0%	0.44 / 0.74	–	0.00310
v2-TR window retrieval	56.1%	0.27 / 0.45	100	0.00208

X. FOLLOW-UP EXPERIMENTS

Retargeting v2. The single start anchor of Sec. V was the main source of loss: every segment began at one robot posture, so most left the workspace and the retargeted joint trajectories covered one posture cluster. We replaced it with multi-seed retargeting under a strict budget: every real-robot quantity (seed bank, workspace gate, posture penalty, trajectory windows) comes from 30 robot episodes only, and results are measured against a disjoint set of 120 robot episodes. Table III compares four variants on the same 1,229 candidate segments. Trajectory-window retrieval (TR), which starts each human segment from the closest feasible robot trajectory window, is the most robot-like: posture distance to the held-out robot data falls from 1.49 to 0.27 rad (median) and start postures span 100 clusters instead of 1. It solves fewer segments than the kinematic variants because it cannot place fast motions; among the 686 segments both TR and the penalised kinematic variant solve, their joint-velocity distributions are similar (W1 0.00207 vs 0.00226), so the posture gain, not a dynamics gain, is what TR adds. The remaining failures are almost all "no workspace-compatible window" (399 of 402 unsolved).

Robot-like re-collection. To test whether the loss is a collection-style problem, we added a robot-like protocol to the collector (30 s takes, live wrist-rotation speed and grasp feedback against robot statistics) and recorded 60 new episodes (10 per order) in one session. Its interpretation rules were fixed before retargeting. Live rotation speed halved (median over-threshold fraction 0.144 to 0.067), but the per-episode TR yield did not change (0.59 vs 0.59) and TR usable rose only from 56.1% to 58.9%; the per-episode correlation between live

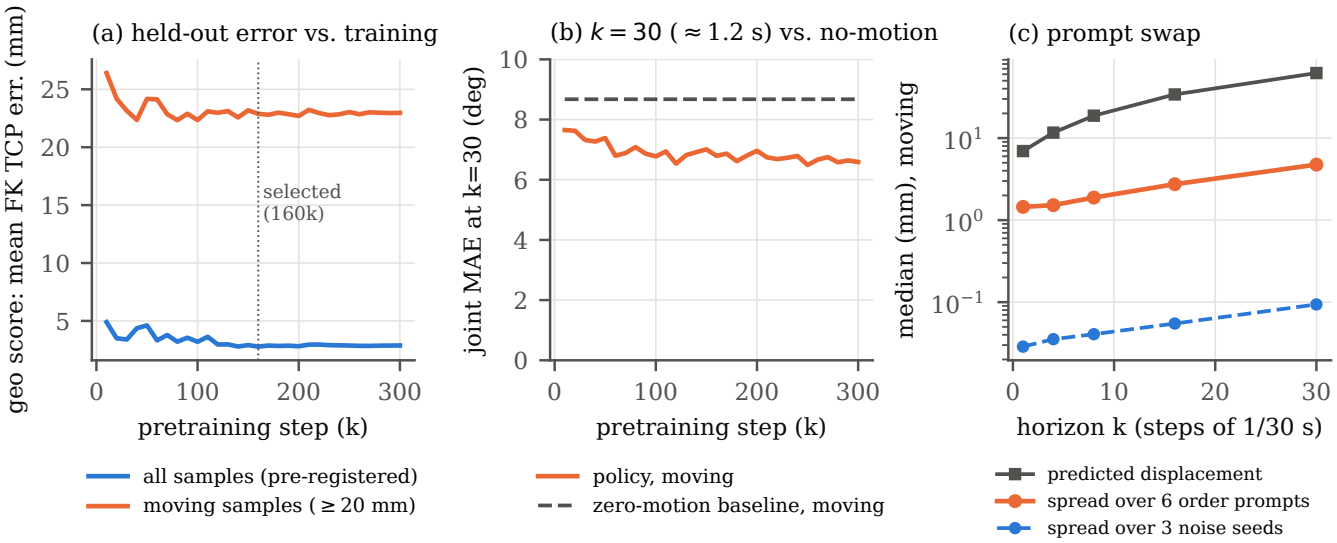


Fig. 4. Offline evaluation. (a) Geo score over training on all versus moving held-out samples; the pre-registered rule selects 160k. (b) Joint MAE at k=30 on moving samples, policy versus zero-motion predictor. (c) Prompt swap on the final checkpoint: prediction spread across the six order instructions (same seed) versus across three noise seeds (true instruction).

robot-likeness and TR yield is weak (Spearman $\rho = -0.11$, $p = 0.05$, pooled $n = 316$). Robot-like movement, as measured live, is associated with, but not a proxy for, retarget feasibility; workspace placement remains the main failure. The final TR dataset combines both collections: 996 segments (896 train / 100 val) from 316 source episodes, with the original 689 segments preserved bit-identically.

Ego pretraining for robot fine-tuning. The first-release ego checkpoint (Sec. VI) and the base checkpoint were each fine-tuned on 150 robot teleoperation episodes of the same task with the same recipe, data and evaluation; only the initialization differs. At the matched step of 250k, the ego-initialized model has a 10.0% lower geo score (episode-level paired bootstrap, 10k draws: 95% CI -14.7 to -5.6%), a 7.2% lower moving-sample geo score (CI -13.4 to -1.8%) and 6.8% lower moving joint MAE at k=30 (CI -12.7 to $+0.5\%$); 8 of the 10 evaluation episodes favour it. The evaluation episodes are part of both fine-tuning sets, so this measures how well each model fits the robot training distribution, not generalization. A smaller-data comparison on 90 robot episodes was stopped early with only two matched steps (10k, 20k); there the signs are mixed (all-sample geo -33% and -11%, moving-sample geo $+2\%$ and $+13\%$ for the ego initialization), so we draw no conclusion from it.

Pretraining on the final dataset. X-VLA was pretrained from `xvla-base` on the 996-segment dataset with the same recipe (planned 300k steps). The run was stopped at 211.7k steps; the 100k and 200k checkpoints are kept, with 100k designated as the initialization for later robot fine-tuning. No robot fine-tuning result from these checkpoints is reported here: the only fine-tuning comparison above uses the first-release ego checkpoint.

XI. Cartesian Ego Data (v3)

Retargeting to robot joints (Secs. V and X) had two costs: the workspace and IK gates removed a third to a half of the candidate motion, and the joint-space target differed from the Cartesian action space our robot policies are fine-tuned in. Version 3 drops the pseudo-joint step. Each human wrist trajectory is converted directly into the same relative end-effector representation as the robot teleoperation data, so that ego and robot samples share one tensor contract and differ only in where they came from.

Contract. Rows are stored at 15 Hz. For row time t the action is a chunk of 16 future poses $A_k = T(t)^{-1}\,T(t+k\Delta)$, $k = 1..16$, with $\Delta = 50.05$ ms (the robot data's step, 0.80 s in total), interpolated on the raw 30 Hz track (linear in position, SLERP in rotation) rather than taken from neighbouring rows. Each arm contributes position, a 6-D rotation and a gripper value (20 dimensions for two arms, "CART20"); the model's 32-D action is padded with 12 dimensions that are exactly zero and never supervised. The state is each arm's pose relative to its own task-start pose plus the two gripper values, the same layout as the robot state. The gripper is the continuous jaw aperture divided by 80 mm; the largest observed value is 0.847, so the "fully open" end is never reached. A row is kept only if both arms are valid at t, all 16 targets lie on valid raw samples no more than 50 ms apart, and a head-camera frame exists within 20 ms; there is no padding. Eleven synthetic tests pin the contract. Fig. 5 shows the wrist trajectories that the labels are cut from; the full videos are in the repository.

Data. The source is the 273 original episodes that have valid per-hand IMU scale and 60 HRL80 episodes; 3 HRL80 episodes are refused because one hand's IMU scale is invalid, leaving 330. Nothing is filtered for robot workspace or IK fea-

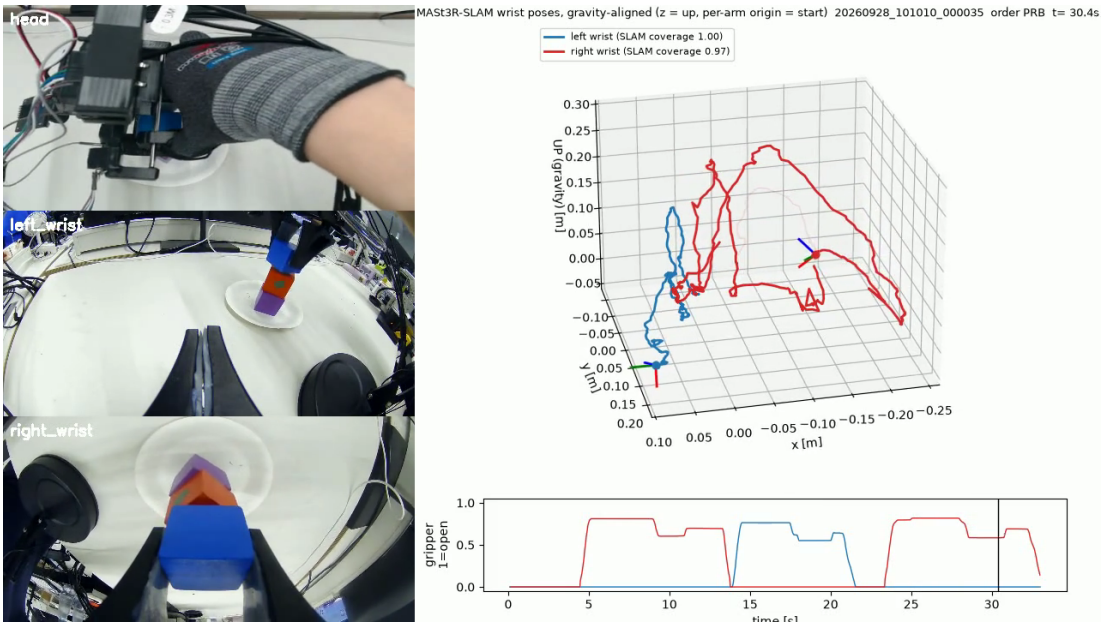


Fig. 5. MASt3R-SLAM wrist trajectories of an HRL80 stacking take (order PRB). Left: head camera and both wrist fisheye cameras. Right: trajectories gravity-aligned (z up), each arm's origin at its start pose, current tool axes drawn (x red, y green, z blue), metric scale from the IMU; below, both grippers (1 = open). Videos of three takes (PRB, RBP, PBR): `docs/media/mast3r_trajectories/hrl80_0000{35,55,60}_wrist.mp4` in the repository.

sibility. Of 111,533 rows at 15 Hz, 57,283 (51.4%) are usable for training; the rest fall on tracking losses, synchronisation gaps or the last 0.8 s of an episode. The split is frozen from the earlier releases (validation = the 26 held-out episodes of Sec. V plus HRL80 episodes 49–54): 298 train episodes with 51,612 rows and 32 validation episodes with 5,671 rows.

Silent pose jumps. Comparing ego and robot state statistics showed per-row state jumps above 100 mm in the ego data that the robot data does not have. Two causes were mixed. First, the exported dataset contains only training rows, so "adjacent" frames can be seconds apart: 1,482 of the 2,598 apparent jumps above 30 mm (57%) are such time gaps, and adjacency has to be decided from row timestamps. Second, the remaining jumps are real: MASt3R-SLAM occasionally moves the camera by 10 cm or more within one 30 Hz frame without flagging tracking loss (55 raw steps faster than 3 m/s in 24 training episodes, none preceded by a loss flag). Because the state is anchored at task start, a one-way jump corrupts every later state of that episode, while the current-anchored action is affected only for 0.8 s. The v2b filter drops the remainder of an episode after a one-way jump (above 100 mm or 45° per row, or an isolated step above 50 mm between two slow steps) and drops isolated spikes only. It acts on 44 events in 42 training episodes and removes one episode entirely; training shrinks to 297 episodes and 48,411 rows (-6.20%), validation is unchanged. The largest step between time-adjacent rows falls from 422.5 to 90.5 mm and steps above 100 mm from 18 to 0; 7 raw steps faster than 3 m/s remain inside valid rows.

How ego data differs from robot data. On 60 ego and 60 robot episodes, with the ego poses attached to a robot start pose: the robot's IK reaches 99.7% (left) and 99.5% (right) of ego positions, and 72.3%/74.6% of full ego poses. The IK configuration used on the robot, which locks one wrist joint, reproduces only 45.2%/52.2% of the robot's *own* recorded poses, a deployment limitation that exists without any human data. Ego hands use a smaller workspace (state p95 0.221/0.214 m versus 0.395/0.369 m) with similar 0.4 s translations (p95 93/99 versus 91/100 mm), and both ego hands are still at the same time far more often (19.5% versus 2.4% of samples at 0.4 s). The largest difference is visual: the variance of the image Laplacian, a sharpness measure, has a median of 1,854/1,487 for the ego wrist fisheye views and 59/64 for the robot's wrist cameras.

XII. Transfer Experiments

All runs in this section share one recipe: X-VLA with the 32-D action head, loss on action dimensions 0–19 only, dimensions 20–31 zeroed at input and output, batch 8, 8-bit AdamW (policy learning rate 10^{-4}, warm-up 1k, cosine decay ending at the last step), bf16 mixed precision, seed 1000. Robot data is R312c, 312 teleoperated episodes (174,012 frames) of the same six-order stacking task. Fine-tuning starts a fresh optimizer and uses robot normalization statistics.

Which soft prompt? X-VLA conditions on a dataset/embodiment id that selects a set of soft prompts. Our earlier runs used id 0. Reading the public base weights shows that only slots 10–17 were trained: in the other 22 slots the encoder and decoder biases are exactly zero and the prompt weights stay near their initialization. Ids 0 and 6, both used by us, were therefore untrained slots. The trained slots are not a better start either: the initial loss on R312c is 1.11 for slot 0, 1.12 for 6, 1.61 for 15, 2.35 for 10, 2.65 for 16, 4.07

TABLE IV
EGO-ONLY CHECKPOINT (40K), MEDIAN DIRECTION COSINE AT k=4/8/16 (LEFT | RIGHT) WHEN IMAGES AND STATE COME FROM DIFFERENT DOMAINS. 100 EGO VALIDATION FRAMES AND 100 ROBOT FRAMES; 21–55 MOVING FRAMES PER CELL.

Images + state	Left k=4/8/16	Right k=4/8/16
ego + ego	0.87 / 0.91 / 0.92	0.89 / 0.94 / 0.84
ego + zero	0.77 / 0.80 / 0.89	0.85 / 0.80 / 0.66
ego + robot	0.76 / 0.81 / 0.90	0.72 / 0.69 / 0.72
robot + robot	0.36 / 0.30 / −0.01	0.19 / 0.02 / −0.15
robot + zero	0.17 / 0.25 / −0.25	0.24 / 0.06 / −0.02
robot + ego	0.16 / 0.24 / −0.16	0.49 / 0.33 / −0.03

for 17 and 22.80 for 11. All runs below use the unused slot 20 as our embodiment's own slot.

Ego-only model on robot frames. After 40k steps on ego data alone, the model was queried, teacher-forced, on 120 robot frames and compared with a model trained 40k steps on robot data (these frames are in that model's training set, so its value is an upper reference). The median cosine between predicted and recorded translation at 0.4 s is $+0.81/+0.77$ (left/right) for the robot model and $-0.05/-0.03$ for the ego model. No combination of a left/right swap and axis sign flips raises the ego model above $+0.16$, and the wrist-camera optical flow has the same sign relations to TCP motion in both domains, so this is not a convention error.

The gap is in the images. Table IV swaps the two inputs. With ego images the model keeps a cosine of 0.69–0.90 whatever state it is given, robot state included; with robot images it falls to between -0.25 and $+0.49$ whatever the state. The failure is visual, which fits the 30-fold sharpness difference of the wrist views.

Ego pretraining as an initialization. We fine-tuned on R312c from the base model (A) and from the ego model after 50k steps (B), with identical data, schedule (150k-step cosine), seed and recipe; both were stopped by us near 60k. At step 200 the training loss is 0.945 for A and 0.244 for B. The ratio B/A is 0.49 over the first 5k steps, 0.875 over 5k–20k and 0.95 over 20k–40k, and over 55k–60k the two are tied (0.0733 versus 0.0745; B lower at 13 of 24 logged points) (Fig. 6). Ego pretraining therefore speeds up early fitting of the robot data and gives no measurable advantage in training loss by 60k. This is training loss with one seed per arm; we have no held-out or closed-loop comparison of A and B. Two 300k runs finished later, but on different robot datasets (R312c for the ego-initialized run, 384 episodes for the scratch run), so they are not a transfer comparison and we do not report one.

Attempts to close the visual gap. To make the ego wrist views look like the robot's, we re-render each fisheye frame as a virtual pinhole camera (62–70° horizontal field of view, principal point shifted so that the jaws sit low in the image as on the robot), then downsample, blur and JPEG-compress it. Labels, state and head images are unchanged. Applied to every frame (ROBOT100) the wrist sharpness median drops from 1,854/1,487 to 181/132, still two to three times the robot's 59/64. A run with 70% of frames re-rendered was stopped at 5k steps; a co-training run with exactly four ego (ROBOT100) and four robot samples per batch was stopped at 88.6k steps without evaluation. A 300k ROBOT100 pretraining run (297 episodes, 48,411 rows, 49.6 epochs) finished on 5 October at a training loss of 0.007; its checkpoints are the ego initializations of Sec. XIII. No offline transfer metric was computed for the re-rendered models.

On the robot. Checkpoints from every run above were executed on the reBot through our deployment interface: 71 checkpoints, 49,431 control cycles and 1,681 run starts between 2 and 4 October. The cycle log records IK tracking of the model's own waypoints, not task outcome, and has no episode or instruction field, so it yields no success rate. Two operator observations were not logged. First, the ego-initialized model at 10k fine-tuning steps grasped cubes but picked the wrong colour. Second, the ego-initialized model fine-tuned on R312c for 300k (B300, checkpoints between 75k and 205k) completed the full three-cube stack on the robot in closed loop. The number of trials was not recorded and no video was taken, so we state only that closed-loop three-cube stacking has been demonstrated, with no success rate. It also says nothing about the value of ego pretraining: no scratch model trained on the same R312c data was evaluated under the same protocol (the finished scratch 300k run used R384). Between 5 and 7 October the only bimanual robot use was one short run of the R384 scratch model (200k, 4 control cycles). A trial logger with the fields listed in Sec. XV was added to the deployment interface on 4 October but has not been used, so there are still no logged trial outcomes. During a storage clean-up on 4 October only checkpoints at multiples of 50k were kept; of the B300 checkpoints that stacked, 150k and 200k survive and the others (155k–205k) are gone, so the demonstration can be re-run only from 150k or 200k.

XIII. LOW-DATA ABLATION: 30 ROBOT EPISODES

The 312-episode comparison of Sec. XII gives the robot data every chance to wash out the initialization. The question that matters for using human data is whether it helps when robot data is scarce. We therefore fine-tuned on R30, episodes 150–179 of R312c (30 episodes, 17,600 frames, five per stacking order), with the recipe of Sec. XII, R312c normalization statistics, seed 1000 and a 600k-step cosine schedule, from three initializations: the base model (scratch), and the ROBOT100 ego pretraining run of Sec. XII at 100k and at 300k steps. The 100k checkpoint is taken from the middle of the same 300k-step schedule, so its learning rate had not decayed; it is not a separate 100k-step run. The two ego arms also ran on different machines (scratch and 100k-init on an RTX 4090 with PyTorch 2.6, 300k-init on an RTX 5090 with PyTorch 2.7.1), so they are not bit-identical twins.

Result. Table V and Fig. 7 show the same pattern as the 312-episode comparison, stretched out. Both ego initializations start at less than half the scratch loss (window ratio 0.54 and 0.49 over 1k–5k). The 100k-step initialization loses its advantage by about 100k steps (ratio 0.97 over 50k–100k,

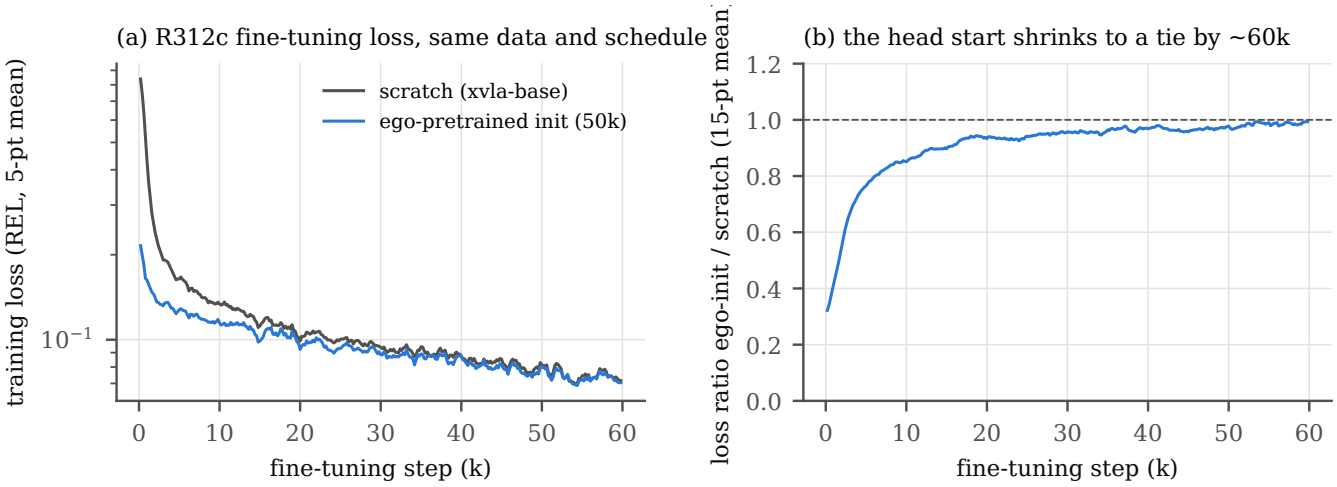


Fig. 6. R312c fine-tuning from the base model versus from the 50k ego checkpoint, same data, schedule and seed. (a) Training loss. (b) Ratio of the two; values below 1 favour the ego initialization. Training loss only.

TABLE V
R30 FINE-TUNING, TRAINING LOSS (200-STEP RUNNING MEAN) AT MATCHED STEPS, AND THE RATIO OF WINDOW MEANS TO SCRATCH (BELOW 1 FAVOURS THE EGO INITIALIZATION). ONE SEED PER ARM; NO HELD-OUT OR ROBOT EVALUATION.

Step	Scratch	Ego-PT 100k init	Ego-PT 300k init
200	0.801	0.239	0.446
1k	0.312	0.103	0.117
10k	0.065	0.056	0.040
50k	0.019	0.017	0.014
100k	0.009	0.009	0.007
Window		ratio	ratio
1k–5k		0.544	0.493
10k–20k		0.823	0.697
50k–100k		0.970	0.856
100k–150k		1.021	0.914

1.02 over 100k–150k); the 300k-step one keeps a smaller advantage longer (0.86 and 0.91) and reaches parity at about 200k. Beyond about 150k all losses are 0.001–0.004 at the printed precision and ratios are not meaningful. The 100k-step initialization starts lower than the 300k-step one (0.239 versus 0.446 at step 200) although it was pretrained less; we have not explained this. All three runs were stopped by us to free GPUs, at 471k (scratch), 305k (100k-init) and 310k (300k-init), after 214, 139 and 141 epochs: with 30 episodes, every arm fits its training data almost exactly long before the schedule ends.

What this does and does not show. Training loss on 30 episodes measures how fast each initialization memorizes the robot data, not how well it acts. Because all arms overfit heavily, the relevant comparison is closed-loop at early and middle checkpoints (roughly 10k–150k), where the initializations still differ; no R30 checkpoint has been run on the robot yet, and there is no held-out robot evaluation. The prediction that this experiment can test is whether the earlier fit of the ego-initialized models turns into stacking at fewer fine-tuning steps; the experiment has not been done.

XIV. LESSONS

Four findings changed how we work, and each was found by an explicit check rather than by a loss curve. (i) A pretrained model's conditioning interface has to be audited, not assumed: two of our embodiment ids pointed at soft prompts that had never been trained. (ii) When human and robot data share a tensor contract, the remaining gap can be localized by swapping inputs between domains; here it is the wrist images, not kinematics or normalization. (iii) Each scale or pose source has a regime in which it fails silently: MASt3R-SLAM on rare one-frame jumps here, and IMU scale on slow motion in Part II. Both were caught only by comparison with an independent reference (robot statistics, a known object), which we now build into every new collection. (iv) Training-loss comparisons of initializations end in a tie whenever the robot data is fitted to near zero, at 312 and at 30 episodes alike; only held-out or closed-loop evaluation at early checkpoints can show a benefit, and the logging for it has to exist before the robot runs, not after.

XV. LIMITATIONS AND FUTURE WORK

Completed: dataset collection and calibration; the pose, retargeting and QA pipeline; 300k-step ego fine-tuning and offline evaluation on held-out episodes; retargeting v2, the robot-like re-collection and the matched 150-episode robot fine-tuning comparison; pretraining on the final 996-segment dataset (stopped at 211.7k steps, checkpoints 100k and 200k). Version 3 added the Cartesian ego dataset and its filters, the soft-prompt audit, the domain-swap diagnostics, the matched initialization comparison up to 60k steps and the robot-like wrist re-rendering. Version 4 added the finished ROBOT100 pretraining and the three-arm R30 ablation (training loss,

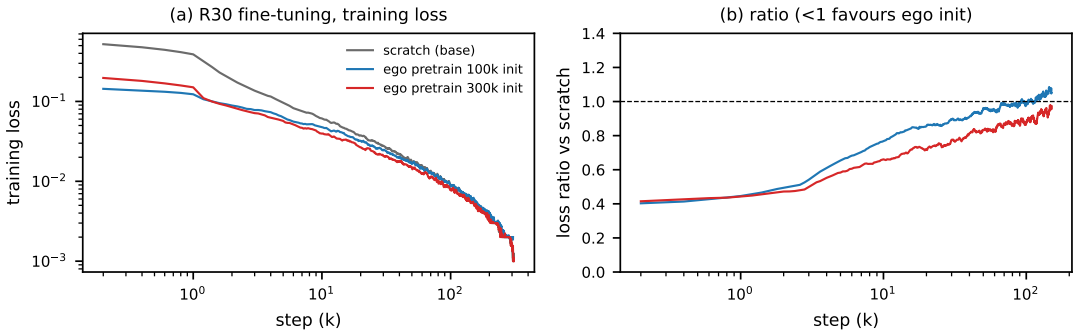


Fig. 7. R30 fine-tuning (30 robot episodes) from the base model and from ego pretraining at 100k and 300k steps. (a) Training loss (log–log, smoothed). (b) Ratio to scratch, shown to 150k; beyond that the 3-decimal losses make the ratio noise. Training loss only, one seed per arm.

stopped at 305k–471k). **Stopped early, no conclusion:** the 90-, 60- and 30-episode robot fine-tuning comparisons of version 2 (at most two matched steps, or only one arm run); the 70%-re-rendered pretraining (5k steps) and the ego/robot co-training run (88.6k steps, never evaluated). **Not yet done:** robot fine-tuning from the 996-segment checkpoints; any robot or held-out evaluation of the R30 arms; a held-out or closed-loop comparison of the ego-initialized and scratch models; outcome logging on the robot (the logger exists but has recorded no trial). The version-2 robot evaluation episodes overlap the fine-tuning data, so a benefit on disjoint robot data remains to be shown.

Not supported by this study: (i) a closed-loop success rate (three-cube stacking was demonstrated once fine-tuned on 312 robot episodes, but trials were not counted), or any closed-loop difference between ego-initialized and scratch models; (ii) generalization to new cube layouts, since layouts were not recorded and absolute geometry is discarded by design; (iii) generalization to unseen stacking orders, since all six orders appear in training; (iv) transfer beyond one operator, one scene and three days; (v) any benefit of human data for robot learning beyond fitting the robot training distribution faster; (vi) that the ego-only policy, or the re-rendered wrist images, transfer to robot cameras. **Planned experiments:** log every robot trial (model, checkpoint, robot data size, layout, instruction, stack level 0–3, wrong colour, failure reason) and run 20–30 trials per model; compare scratch and ego-initialized models at matched fine-tuning steps and at reduced robot data (30 episodes) for the step and data budget at which stacking first succeeds; record layouts from the head camera and hold out layout regions; hold out two orders entirely and test instruction following; train with a static-phase-aware loss or weighting; evaluate closed-loop on the robot with the order instruction as the controlled variable.

More broadly, this work is a first step towards a research agenda in which egocentric human demonstrations provide scalable supervision for robot policies: learning human-to-robot representations that keep task geometry and intent while discarding embodiment detail, and evaluating them with predictive world models before execution. The present results show that the bottleneck is as much in pose recovery, filtering and evaluation design as in the policy.

XVI. Conclusion

We collected 349 bimanual egocentric demonstrations of three-cube stacking, processed them into 561 robot joint-space segments through a gated pipeline, and fine-tuned a 0.9B VLA on the result. Offline, on moving samples the model predicts the direction of held-out human motion reliably (cosine 0.90 at k=30) and reduces joint-space error relative to a zero-motion baseline (by 13–32% across horizons); on static samples it predicts spurious motion, aggregate metrics hide this, and swapping the order instruction does not make predictions more accurate. Multi-seed trajectory retrieval removes much of the retargeting loss, and ego pretraining gives a small, consistent gain when fine-tuning on robot data, measured on the robot training episodes. Converting whole episodes to the robot's Cartesian action contract keeps 330 episodes without any feasibility filter, after a filter for silent SLAM jumps. With that contract the remaining human-to-robot gap is visual: an ego-only model gives no useful action direction on robot camera frames, and the gap follows the wrist images, not the state. As an initialization, ego pretraining speeds up early robot fine-tuning, and the advantage in training loss is gone by 60k steps. An ego-initialized model fine-tuned on 312 robot episodes has completed full three-cube stacks on the robot, so the task itself is solved in closed loop; whether ego pretraining reaches that point with fewer robot episodes or fewer fine-tuning steps is the open question. For slow single-arm motions, a known object replaces IMU scale. Spatial and task-order generalization, a benefit on disjoint robot data, and a measured success rate require the controlled experiments listed above. On 30 robot episodes, ego pretraining again speeds up early fitting, for longer when the pretraining is longer, and again the training-loss advantage disappears; whether it becomes a closed-loop advantage at early checkpoints is the next experiment.

References

[1] C. Chi, Z. Xu, C. Pan, E. Cousineau, B. Burchfiel, S. Feng, R. Tedrake, and S. Song, "Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots," in *RSS*, 2024.

[2] C. Wang, H. Shi, W. Wang, R. Zhang, L. Fei-Fei, and C. K. Liu, "DexCap: Scalable and portable mocap data collection system for dexterous manipulation," in *RSS*, 2024.
[3] S. Kareer *et al.*, "EgoMimic: Scaling imitation learning via egocentric video," *arXiv:2410.24221*, 2024.
[4] C. Wang *et al.*, "MimicPlay: Long-horizon imitation learning by watching human play," in *CoRL*, 2023.
[5] K. Grauman *et al.*, "Ego4D: Around the world in 3,000 hours of egocentric video," in *CVPR*, 2022.
[6] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *RSS*, 2023.
[7] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *RSS*, 2023.
[8] M. J. Kim *et al.*, "OpenVLA: An open-source vision-language-action model," in *CoRL*, 2024.
[9] K. Black *et al.*, "π_0: A vision-language-action flow model for general robot control," *arXiv:2410.24164*, 2024.
[10] J. Zheng *et al.*, "X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model," in *ICLR*, 2026 (arXiv:2510.10274).
[11] R. Cadene *et al.*, "LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch," https://github.com/huggingface/lerobot, 2024.
[12] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *ICLR*, 2023.
[13] R. Murai, E. Dexheimer, and A. J. Davison, "MASt3R-SLAM: Real-time dense SLAM with 3D reconstruction priors," in *CVPR*, 2025.
[14] P. Furgale, J. Rehder, and R. Siegwart, "Unified temporal and spatial calibration for multi-sensor systems," in *IROS*, 2013.
[15] B. Xiao *et al.*, "Florence-2: Advancing a unified representation for a variety of vision tasks," in *CVPR*, 2024.
[16] BrikHMP18 and HandUMI contributors, "HandUMI: Hardware," GitHub repository, https://github.com/murobotics-ai/handumi-hw (first published as `robonet-ai/handumi-hw`), Apache-2.0, 2026. Commit `e58de33` used.
[17] BrikHMP18 and HandUMI contributors, "HandUMI: Software," GitHub repository, https://github.com/murobotics-ai/handumi-sw (first published as `robonet-ai/handumi-sw`), Apache-2.0, 2026. Commit `33cc437` used.
[18] T. Ohkawa, J. Arima, Y. Noguchi *et al.*, "YUBI: Yielding universal bidigital interface for bimanual dexterous manipulation at scale," arXiv:2606.10244, 2026.